МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Специальность системы управления информацией

Выполнил

М. А. Хомич,

студент группы АС-68/24

Проверил

А. А. Крощенко,

доцент Кафедры ИИТ,

«__» _____2026 г.

Брест 2026

Цель: На практике сравнить работу нескольких алгоритмов классификации, таких как метод k-ближайших соседей (k-NN), деревья решений и метод опорных векторов (SVM). Научиться подбирать гиперпараметры моделей и оценивать их влияние на результат.

Вариант 10.

Вариант 10

- `Adult Census Income`
- Предсказать, превышает ли доход человека $50 тыс. в год
- **Задания:**
  1. Загрузите данные, обработайте пропуски и категориальные признаки;
  2. Разделите данные на обучающую и тестовую выборки;
  3. Обучите k-NN, Decision Tree и SVM;
  4. Сравните модели по метрике `precision` для класса ">50K";
  5. Определите, какой алгоритм лучше всего идентифицирует людей с высоким доходом.

Код программы:

```
print("Лабораторная работа №3")
print("Вариант 10 — Adult Census Income")

# %% 1. Загрузка данных
import numpy as np
import pandas as pd
from IPython.display import display

df = pd.read_csv('adult.csv')

display(df.head())
print("Размер датасета:", df.shape)
print("Столбцы:", df.columns.tolist())
print("Пропуски ('?') по столбцам:")
print((df == '?').sum()[lambda s: s > 0])

# %% 2. Обработка пропусков и категориальных признаков, разделение выборок
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# Пропуски в датасете обозначены знаком '?'
df = df.replace('?', np.nan)

# Признаки и целевая переменная: 1 — доход > 50K, 0 — доход <= 50K
X = df.drop(columns=['income'])
y = df['income'].apply(lambda v: 1 if '>50K' in str(v) else 0)

num_cols = X.select_dtypes(include=['int64', 'float64']).columns.tolist()
cat_cols = X.select_dtypes(exclude='number').columns.tolist()
```

```
# Разделение: 75% — обучение, 25% — тестирование
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

# Предобработка: числовые — медиана + стандартизация,
# категориальные — самое частое значение + One-Hot Encoding
preprocessor = ColumnTransformer([
    ('num', Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler())
    ]), num_cols),
    ('cat', Pipeline([
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(handle_unknown='ignore'))
    ]), cat_cols),
])

# fit только на обучающей выборке, чтобы не было утечки данных
X_train_prep = preprocessor.fit_transform(X_train)
X_test_prep = preprocessor.transform(X_test)

print("Обучающая выборка:", X_train_prep.shape)
print("Тестовая выборка:", X_test_prep.shape)
print("Доля класса >50K в train:", round(y_train.mean(), 3))

# %% 3. Обучение моделей k-NN, Decision Tree, SVM
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC

models = {
    "k-NN": KNeighborsClassifier(n_neighbors=5),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "SVM": SVC(kernel='rbf')
}

for name, model in models.items():
    model.fit(X_train_prep, y_train)
    print(f"{name}: обучение завершено")

# %% Метрики базовых моделей (precision для класса >50K — главная)
from sklearn.metrics import classification_report, accuracy_score, precision_score

for name, model in models.items():
    y_pred = model.predict(X_test_prep)

    print(f"\n{name}")
    print(classification_report(
        y_test, y_pred,
```

```
        target_names=['<=50K', '>50K'],
        zero_division=0
    ))
    print("Accuracy:", round(accuracy_score(y_test, y_pred), 3))
    print("Precision (>50K):", round(precision_score(y_test, y_pred, zero_division=0), 3))

# %% 4. Исследование k-NN при разном k
from sklearn.model_selection import cross_val_score

print("Исследование k-NN")
knn_results = []
for k in range(1, 16):
    knn = KNeighborsClassifier(n_neighbors=k)

    # кросс-валидация на обучающей выборке (по precision для >50K)
    cv_prec = cross_val_score(
        knn, X_train_prep, y_train, cv=5, scoring='precision'
    ).mean()

    # precision на тестовой выборке
    knn.fit(X_train_prep, y_train)
    test_prec = precision_score(y_test, knn.predict(X_test_prep), zero_division=0)

    knn_results.append({'k': k, 'CV precision': cv_prec, 'Test precision': test_prec})
    print(f"k = {k:2}: CV precision = {cv_prec:.3f}, test precision = {test_prec:.3f}")

knn_df = pd.DataFrame(knn_results)

# Лучшее k выбираем по кросс-валидации (не по тесту!)
best_k = int(knn_df.loc[knn_df['CV precision'].idxmax(), 'k'])
print("\nЛучшее k по кросс-валидации:", best_k)

# %% 5. Итоговое сравнение моделей по precision для класса ">50K"
best_knn = KNeighborsClassifier(n_neighbors=best_k).fit(X_train_prep, y_train)

final_models = {
    f"k-NN (k={best_k})": best_knn,
    "Decision Tree (max_depth=10)": DecisionTreeClassifier(max_depth=10, random_state=42).fit(X_train_prep,
y_train),
    "SVM (rbf)": models["SVM"],
}

rows = []
for name, model in final_models.items():
    y_pred = model.predict(X_test_prep)
    rows.append({
        'Модель': name,
        'Precision (>50K)': precision_score(y_test, y_pred, zero_division=0),
        'Accuracy': accuracy_score(y_test, y_pred),
    })
    print(f"\n{name}")
    print(classification_report(y_test, y_pred, target_names=['<=50K', '>50K'], zero_division=0))

summary = pd.DataFrame(rows).sort_values('Precision (>50K)', ascending=False)
```

```
print("\nСВОДНАЯ ТАБЛИЦА")
display(summary.round(4))

best_model = summary.iloc[0]['Модель']
print(f"Лучше всего по precision для класса >50K: {best_model}")
```

Результаты работы программы:

```
Лабораторная работа №3
Вариант 10 — Adult Census Income
   age workclass  fnlwgt  ... hours.per.week  native.country income
0   90         ?   77053  ...             40   United-States  <=50K
1   82   Private  132870  ...             18   United-States  <=50K
2   66         ?  186061  ...             40   United-States  <=50K
3   54   Private  140359  ...             40   United-States  <=50K
4   41   Private  264663  ...             40   United-States  <=50K

[5 rows x 15 columns]
Размер датасета: (32561, 15)
Столбцы: ['age', 'workclass', 'fnlwgt', 'education', 'education.num', 'marital.status', 'occupation', 'relationship', 'race',
'sex', 'capital.gain', 'capital.loss', 'hours.per.week', 'native.country', 'income']
Пропуски ('?') по столбцам:
workclass         1836
occupation        1843
native.country     583
dtype: int64
Обучающая выборка: (24420, 104)
Тестовая выборка: (8141, 104)
Доля класса >50K в train: 0.241
k-NN: обучение завершено
Decision Tree: обучение завершено
SVM: обучение завершено

k-NN
              precision    recall  f1-score   support

       <=50K       0.88      0.91      0.89      6181
        >50K       0.67      0.60      0.63      1960

    accuracy                           0.83      8141
   macro avg       0.77      0.75      0.76      8141
weighted avg       0.83      0.83      0.83      8141

Accuracy: 0.832
Precision (>50K): 0.669

Decision Tree
              precision    recall  f1-score   support

       <=50K       0.88      0.87      0.87      6181
        >50K       0.60      0.61      0.60      1960

    accuracy                           0.81      8141
   macro avg       0.74      0.74      0.74      8141
```

```
weighted avg       0.81      0.81      0.81      8141

Accuracy: 0.808
Precision (>50K): 0.599

SVM
              precision    recall  f1-score   support

       <=50K       0.88      0.94      0.91      6181
        >50K       0.75      0.58      0.66      1960

    accuracy                           0.85      8141
   macro avg       0.81      0.76      0.78      8141
weighted avg       0.85      0.85      0.85      8141

Accuracy: 0.852
Precision (>50K): 0.746
Исследование k-NN
k =  1: CV precision = 0.589, test precision = 0.589
k =  2: CV precision = 0.712, test precision = 0.711
k =  3: CV precision = 0.653, test precision = 0.650
k =  4: CV precision = 0.720, test precision = 0.719
k =  5: CV precision = 0.670, test precision = 0.669
k =  6: CV precision = 0.718, test precision = 0.721
k =  7: CV precision = 0.685, test precision = 0.693
k =  8: CV precision = 0.715, test precision = 0.722
k =  9: CV precision = 0.689, test precision = 0.691
k = 10: CV precision = 0.712, test precision = 0.718
k = 11: CV precision = 0.689, test precision = 0.692
k = 12: CV precision = 0.713, test precision = 0.720
k = 13: CV precision = 0.694, test precision = 0.691
k = 14: CV precision = 0.711, test precision = 0.708
k = 15: CV precision = 0.695, test precision = 0.688

Лучшее k по кросс-валидации: 4

k-NN (k=4)
              precision    recall  f1-score   support

       <=50K       0.85      0.94      0.89      6181
        >50K       0.72      0.48      0.58      1960

    accuracy                           0.83      8141
   macro avg       0.79      0.71      0.74      8141
weighted avg       0.82      0.83      0.82      8141


Decision Tree (max_depth=10)
              precision    recall  f1-score   support

       <=50K       0.88      0.92      0.90      6181
        >50K       0.71      0.61      0.66      1960

    accuracy                           0.85      8141
```

macro avg	0.80	0.76	0.78	8141
weighted avg	0.84	0.85	0.84	8141

SVM (rbf)

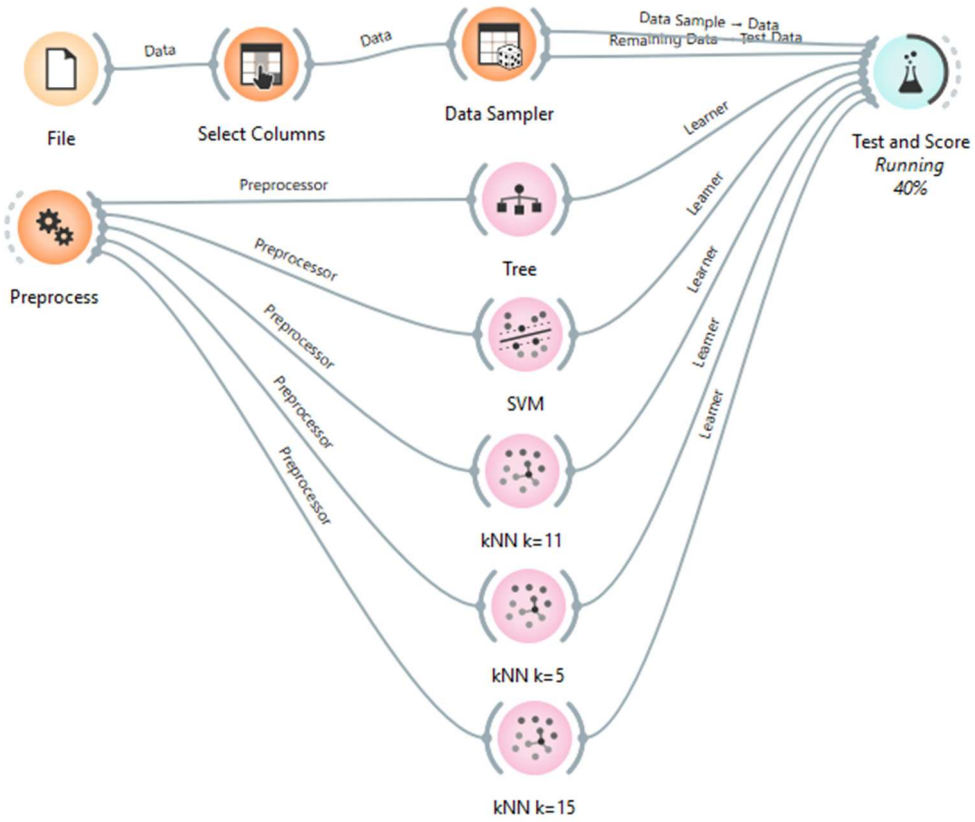
	precision	recall	f1-score	support
<=50K	0.88	0.94	0.91	6181
>50K	0.75	0.58	0.66	1960
accuracy			0.85	8141
macro avg	0.81	0.76	0.78	8141
weighted avg	0.85	0.85	0.85	8141

СВОДНАЯ ТАБЛИЦА

	Модель	Precision (>50K)	Accuracy
2	SVM (rbf)	0.7461	0.8521
0	k-NN (k=4)	0.7194	0.8301
1	Decision Tree (max_depth=10)	0.7130	0.8466

Лучше всего по precision для класса >50K: SVM (rbf)

Результаты моделирования в Orange:



Вывод: В ходе практической работы были изучены и сравнены алгоритмы классификации k-NN, деревья решений и SVM. Были освоены подбор гиперпараметров моделей и оценка их влияния на качество классификации. В результате закреплены практические навыки применения алгоритмов машинного обучения и выбора подходящей модели для решения задач классификации.